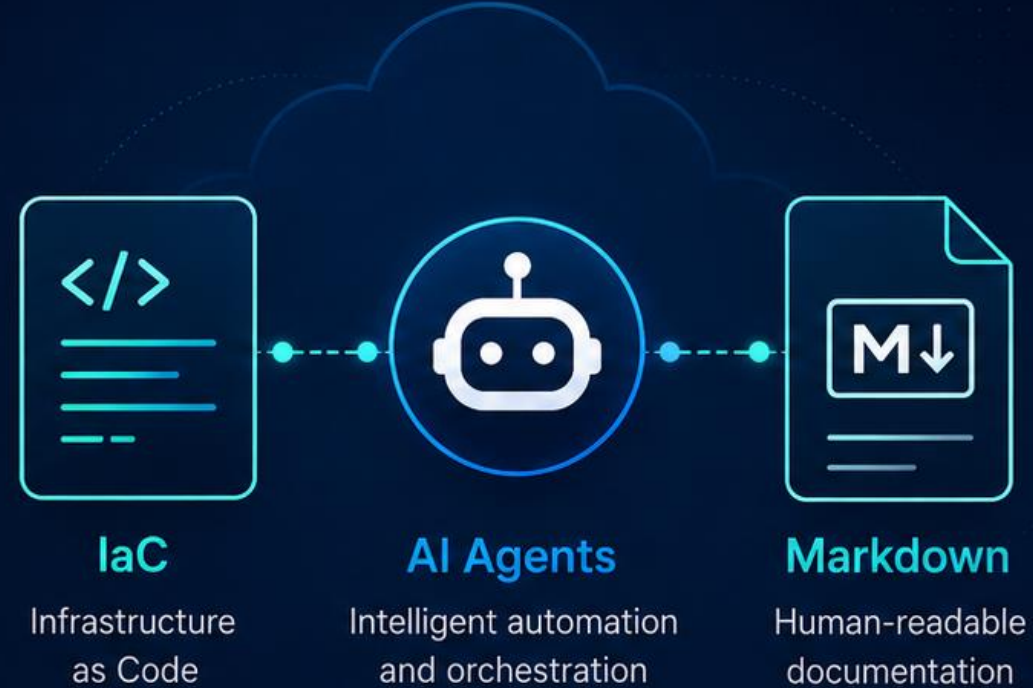


AI Meets Infrastructure

Integrating IaC, AI Agents & Markdown

How the integration works, how it differs from traditional methods, and the key benefits.



1. Introduction



2. How It Works



3. Traditional vs AI



4. Benefits

My Experience Before IaC & AI

What infrastructure work looked like in the traditional approach.



1 Manual provisioning

Servers, networks, and changes were configured by hand.



2 Scattered requirements

Details lived in emails, chats, tickets, and separate documents.



3 Slow change cycles

Even small updates took time to prepare and verify.



4 Environment inconsistency

Settings could differ between dev, test, and production.



5 Knowledge dependency

Progress relied heavily on individual experience and memory.



These challenges created the need for a more **repeatable, automated,** and **well-documented** way of working.

How the Integration Works



i Role of each component

- Markdown = source of intent
- AI Agent = interpreter + automation layer
- IaC = executable infrastructure definition



Human stays in control through review and approval.

End-to-End Workflow



Typical outputs



Terraform /
Bicep files



README and
architecture notes



Change summary



Risk or issue report



From idea to deployment in a **repeatable workflow**.

Traditional Method vs AI-Assisted Method

	 Traditional Approach	 AI-Assisted Approach
 Requirements	Scattered in emails, tickets, docs	Captured clearly in Markdown
 IaC Creation	Manual coding by engineer	Agent drafts code from requirements
 Review	Human-only review	Human review + automated analysis
 Speed	Slower, more repetitive	Faster iteration and updates
 Knowledge Sharing	Dependent on individual expertise	Documented, repeatable, easier onboarding



AI does not replace engineers — it **amplifies** them.

Q&A

Questions & Discussion

— Thank you —

